11.2 服务器启动流程源码分析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160,000 tokens，输出 ~12,000 tokens（本章合计）

理解 Gateway 的启动流程，就是理解"一个 OpenClaw 实例如何从无到有地运行起来"。本节追踪从 `openclaw gateway` 命令到 Gateway 完全就绪的每一步。

11.2.1 入口文件 `src/entry.ts`

一切从 `src/entry.ts` 开始。当用户运行 `openclaw gateway` 时：

```
// src/entry.ts（简化）
process.title = "openclaw";            // 设置进程标题
installProcessWarningFilter();         // 过滤 Node.js 实验性警告
normalizeEnv();                        // 规范化环境变量

// 处理实验性警告抑制
if (!ensureExperimentalWarningSuppressed()) {
  // 解析 CLI Profile 参数（--profile dev 等）
  const parsed = parseCliProfileArgs(process.argv);
  if (parsed.profile) {
    applyCliProfileEnv({ profile: parsed.profile });
  }

  // 动态导入 CLI 主模块并启动
  import("./cli/run-main.js")
    .then(({ runCli }) => runCli(process.argv));
}
```

`ensureExperimentalWarningSuppressed()` 的实现有点意思：Node.js 的 `--disable-warning=ExperimentalWarning` 标志不能通过 `NODE_OPTIONS` 传递（Node.js 的安全限制），所以 OpenClaw 检测到需要时会**重新生成自身**（respawn），在新进程的命令行参数中添加这个标志。通过 `OPENCLAW_NODE_OPTIONS_READY` 环境变量来防止无限递归。

11.2.2 Gateway CLI 启动

CLI 框架使用 **Commander.js**。 `src/cli/program.ts` 导出 `buildProgram()` ，它注册了所有子命令。Gateway 命令在 `src/cli/gateway-cli/register.ts` 中注册。

当用户执行 `openclaw gateway --port 18789` 时，Commander 解析参数后调用 Gateway 启动函数。关键参数包括：

```
--port <number>       # WebSocket/HTTP 端口（默认 18789）
--bind <mode>         # 绑定模式：loopback（默认）| lan | 0.0.0.0
--verbose             # 详细日志
--reset               # 启动前重置会话
--allow-unconfigured  # 允许无配置启动（Docker 场景）
--token <string>      # Gateway 认证 Token
```

11.2.3 服务器启动序列

核心启动逻辑在 `src/gateway/server.impl.ts` 的 `startGatewayServer()` 函数中。这是一个大型异步函数，执行以下步骤：

第一阶段：配置加载与验证

```
// 1. 迁移旧版配置格式
migrateLegacyConfig();

// 2. 加载并验证配置
const cfg = loadConfig();

// 3. 读取配置文件快照（用于热重载比对）
const configSnapshot = readConfigFileSnapshot();

// 4. 解析运行时配置
const runtimeConfig = resolveGatewayRuntimeConfig({
  cfg, port, bind, ...options
});
```

第二阶段：基础设施初始化

```
// 5. 创建子系统日志器
const log = createSubsystemLogger("gateway");
const logChannels = log.child("channels");
const logCron = log.child("cron");
// ...（十几个子日志器）

// 6. 加载插件
const pluginRegistry = loadGatewayPlugins({ cfg, log: logPlugins });

// 7. 初始化子 Agent 注册表
initSubagentRegistry(cfg);

// 8. 加载模型目录
const modelCatalog = await loadGatewayModelCatalog({ cfg });

// 9. 创建运行时状态
const runtimeState = createGatewayRuntimeState({ cfg, runtimeConfig });
```

第三阶段：网络层启动

```
// 10. 解析 TLS 配置
const tlsRuntime = loadGatewayTlsRuntime({ cfg });

// 11. 创建 HTTP(S) 服务器
const httpServer = tlsRuntime
  ? createHttpsServer(tlsRuntime.options)
  : createHttpServer();

// 12. 创建 WebSocket 服务器（挂载在 HTTP 服务器上）
const wss = new WebSocketServer({ noServer: true });

// 13. 绑定端口并监听
httpServer.listen(port, bindAddress);
```

第四阶段：事件与服务绑定

```
// 14. 附加 WebSocket 连接处理器
attachGatewayWsHandlers({
  wss, clients, port,
  resolvedAuth,
  gatewayMethods: listGatewayMethods(),
  events: GATEWAY_EVENTS,
  extraHandlers: coreGatewayHandlers,
  broadcast,
  context,
  // ...
});

// 15. 设置 HTTP 路由
// - OpenAI 兼容 API
// - Open Responses API
// - 控制台 UI 静态文件
// - Canvas/A2UI
// - Webhook 端点
// - Slack HTTP 回调

// 16. 启动 Cron 服务
const cronService = buildGatewayCronService({ cfg, ... });

// 17. 启动配置热重载监听
startGatewayConfigReloader({ cfg, ... });

// 18. 创建通道管理器
const channelManager = createChannelManager({ cfg, ... });
```

第五阶段：侧车服务启动

`src/gateway/server-startup.ts` 中的 `startGatewaySidecars()` 启动附属服务：

```
// 19. 启动浏览器控制服务器（如果启用）
const browserControl = await startBrowserControlServerIfEnabled();

// 20. 启动 Gmail Watcher（如果配置）
await startGmailWatcher(cfg);

// 21. 加载并验证内部钩子
const hooks = await loadInternalHooks({ cfg, ... });

// 22. 启动通道连接
await startChannels();  // WhatsApp、Telegram、Discord、Slack...

// 23. 启动 Tailscale 暴露（如果配置）
await startGatewayTailscaleExposure({ cfg, ... });

// 24. 启动服务发现（Bonjour/mDNS）
await startGatewayDiscovery({ cfg, port, ... });

// 25. 启动心跳
startHeartbeatRunner({ ... });

// 26. 启动维护定时器
startGatewayMaintenanceTimers({ ... });

// 27. 触发 gateway_start 插件钩子
triggerInternalHook(createInternalHookEvent("gateway:start"));
```

第六阶段：启动日志输出

`src/gateway/server-startup-log.ts` 的 `logGatewayStartup()` 打印启动摘要：

```
🦞 OpenClaw Gateway v2026.2.6-3
   Port:     18789
   Bind:     127.0.0.1 (loopback)
   Model:    anthropic/claude-opus-4-6
   Channels: whatsapp, telegram, discord
   Skills:   12 loaded
   Plugins:  memory-core
   Canvas:   :18793
   Ready in: 2.4s
```

11.2.4 服务器实现类

`src/gateway/server.impl.ts` 是整个 Gateway 中最大的文件之一。文件头部的导入列表就有 80 多行，涉及了几乎所有子系统。

它导出两个关键类型：

```
export type GatewayServer = {
  close: () => Promise<void>;
  port: number;
  // ...
};

export type GatewayServerOptions = {
  port: number;
  bind: string;
  verbose: boolean;
  // ...
};
```

`startGatewayServer()` 返回一个 `GatewayServer` 对象，包含关闭方法和运行时元数据。这使得 Gateway 可以被程序化地启动和停止——在 E2E 测试中尤为重要。

子系统日志器的创建模式值得注意：

```
const log = createSubsystemLogger("gateway");
const logCanvas = log.child("canvas");
const logDiscovery = log.child("discovery");
const logTailscale = log.child("tailscale");
const logChannels = log.child("channels");
const logBrowser = log.child("browser");
const logHealth = log.child("health");
const logCron = log.child("cron");
const logReload = log.child("reload");
const logHooks = log.child("hooks");
const logPlugins = log.child("plugins");
const logWsControl = log.child("ws");
```

每个子系统都有自己的日志器，通过 `.child()` 创建子日志器。这样日志输出可以按子系统过滤，调试时非常方便。